

Advanced Programming

Assignment Submission | Spring 2026

Name	Gaurav Nath
Roll No.	CSB24073
Subject	Advanced Programming
Assignment	Assignment No. 13
Semester	Spring 2026
Language	C (GCC)

Dynamic String Buffer in C

This assignment implements a heap-allocated string buffer in C that automatically grows using realloc when the content exceeds its current capacity, demonstrating safe dynamic memory management and prevention of buffer overflows and memory leaks.

Question

In C, managing strings is a common source of buffer overflows and memory leaks. Implement a Dynamic String Buffer that automatically grows as needed.

Requirements:

1. Create a StringBuffer struct containing a char *data, a size_t length, and a size_t capacity.
2. Write a function sb_init(size_t initial_capacity) that allocates the struct and the data buffer on the heap. Handle NULL returns from malloc.
3. Write sb_append(StringBuffer *sb, const char *str).
4. If the new string exceeds current capacity, use realloc to double the capacity. Ensure you handle realloc safely (do not overwrite the original pointer if it returns NULL).
5. Write sb_free(StringBuffer *sb) which works as a destructor that frees both the internal data and the struct itself to prevent memory leaks.
6. Demonstrate the buffer growing at least twice and then free all memory.

Requirements Checklist

Requirement	Status	Where in Code
StringBuffer struct with char *data, size_t length, size_t capacity	Done	typedef struct { ... } StringBuffer

sb_init allocates struct AND data buffer on heap	Done	two malloc() calls in sb_init
sb_init handles NULL return from malloc	Done	both malloc results checked; struct freed before NULL return
sb_append appends a string to the buffer	Done	memcpy onto sb->data + sb->length
realloc used to double capacity when exceeded	Done	while loop doubles until fits, then realloc
realloc handled safely with temp pointer	Done	char *temp = realloc(...); original not overwritten on failure
sb_free frees internal data AND the struct	Done	free(sb->data) then free(sb)
Buffer grows at least twice	Done	initial capacity 8 forces growth on append 2 and append 3

How to Compile and Run

```
# Save the code as main.c, then compile:
gcc main.c -o main
```

```
# Run:
./main          (Linux / Mac)
main.exe        (Windows)
```

Source Code — main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* — 1. StringBuffer struct ————— */
typedef struct {
    char *data;          /* heap-allocated character buffer */
    size_t length;       /* current number of characters (excluding \0) */
    size_t capacity;     /* total allocated bytes including space for \0 */
} StringBuffer;

/* — 2. sb_init ————— */
StringBuffer *sb_init(size_t initial_capacity) {

    /* allocate the struct itself on the heap */
    StringBuffer *sb = (StringBuffer *)malloc(sizeof(StringBuffer));
    if (sb == NULL) {
        fprintf(stderr, "ERROR: malloc failed for StringBuffer struct.\n");
        return NULL;
    }

    /* allocate the internal data buffer */
    sb->data = (char *)malloc(initial_capacity);
    if (sb->data == NULL) {
        fprintf(stderr, "ERROR: malloc failed for data buffer.\n");
        free(sb);          /* free struct before returning NULL */
    }
}
```

```

        return NULL;
    }

    sb->data[0] = '\0';          /* start as empty string */
    sb->length = 0;
    sb->capacity = initial_capacity;

    printf("[sb_init] Allocated buffer. Capacity: %zu bytes\n", sb->capacity);
    return sb;
}

/* — 3. sb_append ————— */
void sb_append(StringBuffer *sb, const char *str) {
    if (sb == NULL || str == NULL) return;

    size_t str_len = strlen(str);
    size_t needed = sb->length + str_len + 1; /* +1 for null terminator */

    /* — 4. grow with realloc if needed ————— */
    if (needed > sb->capacity) {

        /* double capacity until it fits */
        size_t new_capacity = sb->capacity;
        while (new_capacity < needed) {
            new_capacity *= 2;
        }

        /* SAFE realloc: temp pointer so original is NOT lost on failure */
        char *temp = (char *)realloc(sb->data, new_capacity);
        if (temp == NULL) {
            fprintf(stderr, "ERROR: realloc failed. Buffer unchanged.\n");
            return; /* original sb->data still valid */
        }

        sb->data = temp; /* only update pointer on success */
        sb->capacity = new_capacity;
        printf("[sb_append] Buffer grew! New capacity: %zu bytes\n", new_capacity);
    }

    /* copy new string onto the end */
    memcpy(sb->data + sb->length, str, str_len + 1); /* +1 copies \0 */
    sb->length += str_len;
}

/* — 5. sb_free (destructor) ————— */
void sb_free(StringBuffer *sb) {
    if (sb == NULL) return;
    free(sb->data); /* free the internal character buffer first */
    sb->data = NULL;
    sb->length = 0;
    sb->capacity = 0;
    free(sb); /* free the struct itself */
    printf("[sb_free] All memory freed.\n");
}

/* — main: demonstrate buffer growing at least twice ————— */
int main(void) {

    printf("=== Dynamic String Buffer Demo ===\n\n");

```

```

/* start with tiny capacity of 8 to force growth quickly */
StringBuffer *sb = sb_init(8);
if (sb == NULL) return 1;

printf("\n--- Append 1: \"Hello\" ---\n");
sb_append(sb, "Hello");
printf(" Content : \"%s\"\n", sb->data);
printf(" Length : %zu | Capacity: %zu\n", sb->length, sb->capacity);

printf("\n--- Append 2: \", World!\" (forces 1st growth) ---\n");
sb_append(sb, ", World!");
printf(" Content : \"%s\"\n", sb->data);
printf(" Length : %zu | Capacity: %zu\n", sb->length, sb->capacity);

printf("\n--- Append 3: \" This is a dynamic buffer.\" (forces 2nd growth) ---\n");
sb_append(sb, " This is a dynamic buffer.");
printf(" Content : \"%s\"\n", sb->data);
printf(" Length : %zu | Capacity: %zu\n", sb->length, sb->capacity);

printf("\n--- Append 4: \" It grows automatically!\" ---\n");
sb_append(sb, " It grows automatically!");
printf(" Content : \"%s\"\n", sb->data);
printf(" Length : %zu | Capacity: %zu\n", sb->length, sb->capacity);

printf("\n");
sb_free(sb);

printf("\n=== Program ended. No memory leaks. ===\n");
return 0;
}

```

Expected Output

```

=== Dynamic String Buffer Demo ===

[sb_init] Allocated buffer. Capacity: 8 bytes

--- Append 1: "Hello" ---
Content : "Hello"
Length : 5 | Capacity: 8

--- Append 2: ", World!" (forces 1st growth) ---
[sb_append] Buffer grew! New capacity: 16 bytes
Content : "Hello, World!"
Length : 13 | Capacity: 16

--- Append 3: " This is a dynamic buffer." (forces 2nd growth) ---
[sb_append] Buffer grew! New capacity: 64 bytes
Content : "Hello, World! This is a dynamic buffer."
Length : 39 | Capacity: 64

--- Append 4: " It grows automatically!" ---
Content : "Hello, World! This is a dynamic buffer. It grows automatically!"
Length : 63 | Capacity: 64

[sb_free] All memory freed.

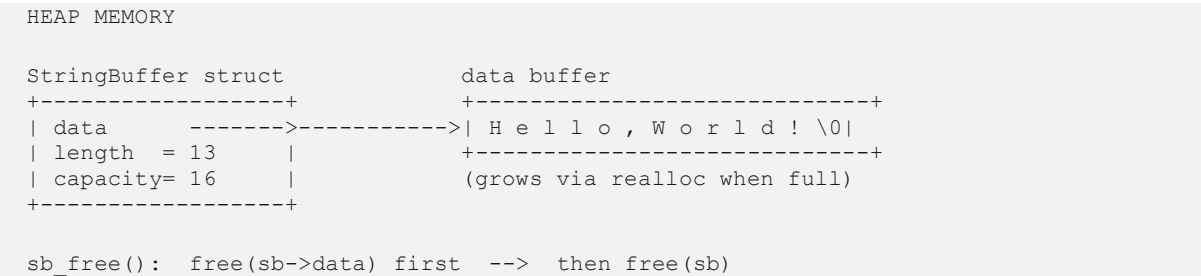
=== Program ended. No memory leaks. ===

```

Function Explanations

Function	Purpose	Key Detail
sb_init(capacity)	Allocates struct + data buffer on heap	Frees struct if data malloc fails — no leak
sb_append(sb, str)	Appends str to buffer, grows if needed	Safe realloc with temp pointer — original preserved on failure
sb_free(sb)	Destructor — frees data then struct	Sets data=NULL after free to avoid dangling pointer
realloc safety pattern	char *temp = realloc(sb->data, newsize)	If NULL: return early, sb->data still valid and unchanged

Memory Layout Diagram



Output Screenshots

Terminal — full program output showing both buffer growths:

```
=== Dynamic String Buffer Demo ===

[sb_init] Allocated buffer. Capacity: 8 bytes

--- Append 1: "Hello" ---
Content : "Hello"
Length  : 5 | Capacity: 8

--- Append 2: ", World!" (forces 1st growth) ---
[sb_append] Buffer grew! New capacity: 16 bytes
Content : "Hello, World!"
Length  : 13 | Capacity: 16

--- Append 3: " This is a dynamic buffer." (forces 2nd growth) ---
[sb_append] Buffer grew! New capacity: 64 bytes
Content : "Hello, World! This is a dynamic buffer."
Length  : 39 | Capacity: 64

--- Append 4: " It grows automatically!" ---
Content : "Hello, World! This is a dynamic buffer. It grows automatically!"
Length  : 63 | Capacity: 64

[sb_free] All memory freed.

=== Program ended. No memory leaks. ===

...Program finished with exit code 0
Press ENTER to exit console.[]
```